

IMPACTOS
BUSINESS OPERATIONS

Operations OS

Engineering Specification

Version 1.0

Document Status: Draft for Engineering Implementation

Audience: Future Studio Engineering Team • AI Development Team • Product Owner • Technical Leads • QA Team

Editor's Note

This edition consolidates and formats all specification material provided to date. All five modules are now present in full.

Table of Contents

1. Introduction

Operations OS is the first Business Operating System within ImpactOS.

It is the operational foundation upon which Future Studio manages its internal activities, staff collaboration, projects, accountability, reporting, and execution.

Unlike traditional task management applications, Operations OS is designed to function as the organization's daily operating environment. Every staff member, intern, supervisor, department lead, and administrator interacts with Operations OS every working day. It is the place where work begins, progresses, is reviewed, reported, and improved.

Every other Business Operating System — including Program OS, Venture OS, Investor OS, CRM & Relationship OS, Ecosystem OS, and Reporting & Business Intelligence — depends on Operations OS functioning correctly.

Build Order: Operations OS must be completed, stabilized, and validated before significant development continues on the remaining Business Operating Systems.

2. Business Objective

The objective of Operations OS is to provide Future Studio with a centralized environment where every member of the organization can:

- Organize daily work.
- Create and receive tasks.
- Collaborate with teammates.
- Participate in projects.
- Plan weekly work.
- Submit weekly reports.
- Raise operational blockers.
- Monitor deadlines.
- Track progress.
- Communicate internally.
- Maintain organizational accountability.

Operations OS should eliminate the need for multiple disconnected operational tools by providing one integrated workspace.

3. Engineering Team Instructions

This document is the official implementation specification for Operations OS. Before writing any code, every engineer — AI or human — must follow the Future Studio Engineering Development Protocol.

First Principle: Your first responsibility is not to build. Your first responsibility is to understand.

For every module:

- Review the existing implementation.
- Test the current workflow.
- Compare existing behaviour with this specification.
- Identify completed functionality.
- Identify partially completed functionality.
- Identify missing functionality.
- Extend existing functionality instead of rebuilding it.
- Preserve backward compatibility.
- Perform regression testing before submission.
- Do not redesign the application's UI unless specifically instructed.

Reuse existing:

- APIs
- Components
- Database tables
- Services
- Utility functions
- Global styling
- Existing business logic

Avoid duplication wherever possible.

4. Global Engineering Standards

These standards apply to every module within Operations OS.

4.1 Multilingual Support (Mandatory)

Every page, modal, component, notification, email template, validation message, button label, and user-facing interface must support both English and French. A feature is not considered complete until both language versions have been implemented.

4.2 Standard Date Fields

Every task and subtask must use consistent terminology. The standard fields are Start Date and End Date. These fields must synchronize automatically with the Calendar module.

Do not use: “Finish Date,” “Due Date,” or “Deadline” unless explicitly required by another business workflow.

4.3 Existing User Interface

Do not redesign the existing application. Continue using existing layouts, reusable components, the design system, typography, spacing, colour palette, and utility components. Maintain visual consistency across the platform.

4.4 Existing Architecture

- Reuse existing APIs.
- Extend existing services.
- Reuse database structures.
- Reuse shared components.
- Extend business logic.
- Create new structures only when no suitable implementation exists.

4.5 Backward Compatibility

Every implementation must preserve existing functionality. No existing workflow should break because of a new implementation. Regression testing is mandatory.

5. Operations OS Overview

Operations OS consists of five integrated modules. These modules should not be treated as isolated software components. Each module represents one complete operational workflow. Developers assigned to a module should understand how their implementation affects the remaining modules.

Module 1 Personal Work Management	Purpose: The individual's daily operational workspace. Includes: • Tasks & subtasks, assignment, calendar sync, notifications, attachments, comments
Module 2 Weekly Accountability	Purpose: The weekly planning, review, and reporting cycle. Includes: • Weekly Standups, Weekly Retros, blockers, carry-over, reports, metrics
Module 3 Project Operations	Purpose: Multi-person collaborative execution. Includes: • Project creation, team management, project tasks, timeline, progress, reporting
Module 4 Communication & Collaboration	Purpose: Contextual internal communication. Includes: • Direct messaging, task/project discussions, broadcasts, announcements, notifications
Module 5 Operational Administration	Purpose: Oversight, governance, and reporting. Includes: • Team management, internal reporting, metrics, finance, document management

Each module has its own business workflow but remains tightly integrated with the others.

6. Complete Business Workflow

Operations OS follows one continuous operational cycle. The workflow begins on Monday and repeats every operational week.

Monday

The user logs into ImpactOS. The system checks whether the user already has a Weekly Standup for the current week.

- If one exists: the existing Standup opens.
- If none exists: the user is prompted to create a new Weekly Standup.

Business Rule: One User = One Weekly Standup = One Operational Week. Multiple Standups cannot exist for the same user during the same week.

Weekly Planning

Inside the Weekly Standup the user begins planning work. The user may:

- Create Tasks
- Create Subtasks
- Assign Tasks
- Assign Subtasks
- Set Priorities
- Set Categories
- Select Start Date
- Select End Date

Every task created during that week belongs to that week's Standup.

Task Assignment

A task may remain personal, or it may be assigned to another authorized user. Assignments should respect group permissions, project permissions, and user permissions.

The receiving user must Accept or Decline the assignment. Until accepted, the assignment remains pending.

During the Week

Throughout the week users may:

- Edit Tasks
- Create additional Tasks
- Create additional Subtasks
- Update progress
- Add comments

- Collaborate
- Attach documents
- Attach URLs
- Update scheduling

Task creation is never restricted to Monday. Users may continue planning throughout the week.

Weekly Review

Whenever the user wishes to review weekly progress, they navigate to Weekly Retro. The Weekly Retro displays every task created under that week's Standup. Each task is reviewed individually. For every task the user may Mark Task Completed or Raise a Blocker — the only two primary actions during the review process.

Blocker Workflow

If the user cannot complete a task, they should not mark it completed. Instead they select Add Blocker.

Implementation Note: The engineering team should convert the current blocker implementation into a dedicated Blocker action within the Weekly Retro.

Selecting Add Blocker opens a modal. The modal should capture:

- Blocker Title
- Description
- Reason
- Priority
- Optional Reference URL
- Additional Notes

After submission, the blocker becomes attached to the task, the task remains incomplete, the Super Admin receives notification, and the responsible supervisor receives notification. The blocker remains active until resolved.

Business Rule: Only the user who originally created the blocker may mark the blocker as resolved. No other user may close that blocker.

Resolving Blockers

When the underlying issue has been resolved, the original blocker owner returns to the Weekly Retro. The blocker now displays Mark Blocker as Resolved. Only after this action is completed can the related task be marked completed. If the blocker remains active, the task remains incomplete.

Completing Tasks

When marking a task completed, the system first checks for active blockers. If no blockers exist, the task is completed immediately. If active blockers exist, the system displays each blocker individually and the user must confirm whether each blocker has been resolved. If any blocker remains unresolved, task completion is prevented.

Parent Tasks

Parent Tasks may contain multiple Subtasks. Example:

- Task: Permission Engine
- Subtask: Database Migration ✓
- Subtask: API Development ✓
- Subtask: UI Improvements (Blocked)
- Subtask: Testing

Individual subtasks may be completed independently. However, the Parent Task cannot be completed until every required Subtask has been completed and every active Blocker has been resolved. This ensures accurate operational reporting.

Module 1 — Personal Work Management

The foundation of Operations OS.

Introduction

Personal Work Management is the foundation of the Operations OS.

Every user within Future Studio — including staff, interns, supervisors, managers, department heads, and administrators — interacts with this module every working day.

Its purpose is to provide every individual with a centralized workspace where they can organize, plan, execute, monitor, and report their daily work. This module is not simply a task management system — it represents the operational workspace of every user.

Everything a user performs throughout the week begins here before flowing naturally into Weekly Accountability, Project Operations, Communication & Collaboration, and Operational Administration. Every task, subtask, assignment, schedule, notification, and activity created within this module contributes automatically to the overall operational workflow of ImpactOS.

Foundation Principle: Personal Work Management should always be treated as the operational foundation of Operations OS.

Business Objective

The objective of Personal Work Management is to provide every user with a structured and intuitive workspace where they can:

- Organize their work.
- Plan their weekly activities.
- Create personal tasks.
- Create subtasks.
- Assign work to authorized users.
- Receive assigned work.
- Accept or decline assigned work.
- Schedule activities.
- Track progress.
- Collaborate with teammates.
- Manage deadlines.
- Receive operational notifications.

The system should minimize duplicate work while maximizing visibility, accountability, and productivity.

Engineering Team Instructions

This module already exists within the current platform. Your responsibility is not to redesign the interface.

Before implementing any changes:

- Review the current implementation.
- Test every existing workflow.
- Identify completed functionality.
- Identify partially completed functionality.
- Identify missing functionality.
- Compare the existing implementation against this specification.
- Extend existing functionality wherever possible.
- Preserve backward compatibility.

Do not duplicate APIs, components, services, database tables, business logic, or utility functions. Reuse the existing architecture wherever possible. Every new implementation must support both English and French before it is considered complete.

Relationship With Other Modules

Relationship with Module 2 — Weekly Accountability

Every task created within Personal Work Management automatically belongs to the user's Weekly Standup. Tasks reviewed during the Weekly Retro originate from this module. Carry-over Tasks originate from this module. Weekly Reports use operational data generated from this module.

Relationship with Module 3 — Project Operations

A task may be either a Personal Task or a Project Task. When a Project Task is assigned to a user, it automatically becomes part of that user's Personal Work Management workspace. The user should never recreate the same task manually.

Relationship with Module 4 — Communication & Collaboration

Every task should support collaboration. Users should be able to discuss tasks, mention users, upload attachments, and share supporting links. Communication remains attached to the relevant task.

Relationship with Module 5 — Operational Administration

Operational reports, productivity metrics, and management dashboards are automatically generated using data collected from this module. Users should never submit duplicate reports for work already recorded.

Business Case Study

The following case study represents the agreed Future Studio operational workflow. Developers should implement this workflow exactly as described.

Monday Morning

Sarah joins Future Studio as a Developer Intern. Before accessing Operations OS she has already:

- Registered through her assigned Group Registration Link.
- Been approved by the supervisor.
- Activated her account.
- Updated her personal profile.

- Been assigned to the Engineering Interns group.
- Been granted access to the Engineering sidebar.
- Been assigned to Project Alpha.

When Sarah logs into ImpactOS, she lands on her Personal Dashboard. Her dashboard displays:

- Current Weekly Standup
- My Tasks
- Assigned Tasks
- My Calendar
- Recent Notifications
- Assigned Projects
- Upcoming Start Dates
- Upcoming End Dates
- Recent Activity

Because this is Monday and no Weekly Standup exists yet, she sees Create Weekly Standup. After creating it, the Weekly Standup becomes her operational workspace for the remainder of the week.

Planning the Week

Sarah begins planning her work. She creates a new task: “Implement Permission Profiles.” She enters a Title, Description, Category, Priority, Start Date, and End Date. The task immediately appears on her dashboard, inside this week’s Weekly Standup, on her calendar, and in her personal task list.

Creating Subtasks

Sarah divides the work into manageable pieces: Review Existing Permissions, Design Permission Profiles, Database Migration, API Development, and UI Testing. Each subtask remains connected to the parent task.

Assigning Work

Sarah decides David should perform API Testing. She assigns the subtask to David. David immediately receives a Pending Assignment and must explicitly choose to Accept or Decline. If David declines, Sarah receives a notification. If David accepts, the subtask automatically becomes part of David’s workload. Assignment history must always be preserved.

Working Throughout the Week

During the week Sarah may create additional tasks and subtasks, edit existing tasks, change priority, Start Date, or End Date, upload files, attach supporting URLs, add comments, and mention teammates. Users are not restricted to planning only on Monday — new work may be added throughout the week.

Calendar Synchronization

Whenever Sarah updates the Start Date or End Date, the Calendar updates automatically. Users should never manually create calendar entries for operational work.

Notifications

The system automatically notifies users when a task is assigned, an assignment is accepted or declined, a task approaches or passes its End Date, a comment is added, a user is mentioned, or a subtask is assigned. Notifications must support both English and French.

Business Rules

1. Every user has one Personal Workspace.
2. Tasks may be created at any time during the operational week.
3. Tasks may contain unlimited subtasks.
4. Tasks may be assigned only to authorized users.
5. Assigned users must accept or decline assigned work.
6. Assignment history must be preserved.
7. Every task belongs to exactly one Weekly Standup.
8. Every task must include Title, Description, Category, Priority, Start Date, and End Date.
9. Changes to Start Date or End Date automatically update the Calendar.
10. Users may upload files and attach reference URLs.
11. Every task supports comments and mentions.
12. All user-facing interfaces, validation messages, notifications, and emails must support English and French.

Feature Breakdown

1.1 Personal Dashboard	Purpose: Provide each user with a personalized operational workspace. Includes: • Current Weekly Standup • My Tasks • Assigned Tasks • Calendar Summary • Notifications • Assigned Projects • Recent Activity
1.2 Personal Task Management	Purpose: Allow users to manage their personal work. Includes: • Create / Edit / Archive / Delete (permission controlled) Task • Duplicate Task • Task History
1.3 Subtask Management	Purpose: Break larger tasks into manageable activities. Includes: • Create / Edit / Delete / Complete Subtask • Parent Task Relationship
1.4	Purpose: Assign work to other authorized users.

Task Assignment	Includes: • Assign Task / Subtask • Accept / Decline Assignment • Assignment History • Reassignment
1.5 Categories & Priorities	Purpose: Organize work consistently. Includes: • Configurable Categories • Priority Levels: Critical, High, Medium, Low
1.6 Calendar Integration	Purpose: Synchronize operational work with the calendar. Includes: • Start Date • End Date • Automatic Calendar Updates
1.7 Notifications	Purpose: Keep users informed. Includes: • Assignment / Acceptance / Decline Notifications • Due Date Reminders • Overdue Notifications • Mentions • Comments
1.8 Attachments & Reference Links	Purpose: Provide supporting resources for work. Includes: • File Upload (images, documents) • Reference URLs — e.g. GitHub, Google Docs, Figma, Vercel, Jira
1.9 Comments & Activity History	Purpose: Maintain collaboration history. Includes: • Comments • Mentions • Activity Timeline • Edit History

Developer Tickets

This module should be implemented as a single engineering sprint. The following tickets are closely related and should be developed together to maintain business consistency.

Ticket 1.1 — Personal Dashboard

Review and improve the existing dashboard so it becomes the user's primary operational workspace. Reuse existing widgets wherever possible. Do not redesign the interface.

Ensure the dashboard displays:

- Current Weekly Standup
- Assigned Tasks

- Personal Tasks
- Upcoming Start/End Dates
- Calendar Summary
- Notifications
- Assigned Projects
- Recent Activity

Acceptance Criteria

- ☐ Dashboard loads correctly.
- ☐ Dashboard only displays information relevant to the logged-in user.
- ☐ Counts update automatically.
- ☐ English and French supported.
- ☐ Existing functionality remains operational.

Ticket 1.2 — Personal Task Management

Improve the existing task workflow.

Users should be able to:

- Create, edit, delete (permission controlled), archive, and duplicate tasks.
- View task history.
- Each task must contain: Title, Description, Category, Priority, Start Date, End Date, Status, Assigned Users, Parent Project (optional).

Acceptance Criteria

- ☐ Users can create, edit, delete, archive, and view history of tasks.
- ☐ Task validation functions correctly.
- ☐ Start Date and End Date synchronize with Calendar.
- ☐ English and French translations exist.

Ticket 1.3 — Subtask Management

Every task should support unlimited subtasks. Subtasks should inherit the parent task where appropriate.

Each subtask should support:

- Title, Description, Assigned User, Priority, Start Date, End Date
- Attachments
- Comments

Acceptance Criteria

- ☐ Users can create, edit, delete, and independently complete subtasks.
- ☐ Parent task progress updates correctly.

Ticket 1.4 — Task Assignment Workflow

Improve the assignment workflow. No task should automatically become another user's responsibility without acceptance.

Acceptance Criteria

- ☐ Assignment notification works.
- ☐ Accept workflow works.
- ☐ Decline workflow works.
- ☐ Assignment history is preserved.

Ticket 1.5 — Categories & Priorities

Review existing implementation. Categories should remain configurable (e.g. Engineering, Finance, Marketing, Administration, Research). Priority options: Critical, High, Medium, Low.

Acceptance Criteria

- ☐ Categories work.
- ☐ Priorities work.
- ☐ Filtering works.
- ☐ Sorting works.

Ticket 1.6 — Calendar Synchronization

Calendar should synchronize automatically whenever Start Date or End Date changes. Users should never create calendar entries manually.

Acceptance Criteria

- ☐ Calendar updates automatically.
- ☐ Changes appear immediately.
- ☐ English and French supported.

Ticket 1.7 — Notifications

Review existing notification system.

Notifications should exist for:

- Task Assignment
- Assignment Accepted / Declined
- Upcoming End Date
- Overdue Task
- Comments
- Mentions

Acceptance Criteria

- ☐ Notifications deliver correctly.
- ☐ Notifications link to the relevant task.
- ☐ French translations implemented.

Ticket 1.8 — Attachments

Users should be able to upload files, images, and documents, and add reference URLs (e.g. GitHub, Figma, Google Docs, Vercel, Linear, or any valid URL).

Acceptance Criteria

- ☐ File upload works.
- ☐ Reference URLs work.
- ☐ Attachments remain linked to task.

Ticket 1.9 — Comments

Every task should support comments. Comments should record author and timestamp, support mentions, and preserve edit history.

Acceptance Criteria

- ☐ Comments work.
- ☐ Mentions notify users.
- ☐ History preserved.

Module 1 Self-Testing Checklist

Before submitting this module, engineering must verify:

Dashboard

- ☐ Dashboard loads correctly.
- ☐ Dashboard statistics update automatically.
- ☐ Only user-specific information is displayed.

Tasks

- ☐ Create Task
- ☐ Edit Task
- ☐ Delete Task
- ☐ Archive Task
- ☐ Duplicate Task

Subtasks

- ☐ Create Subtask
- ☐ Edit Subtask
- ☐ Delete Subtask
- ☐ Complete Subtask

Assignment

- ☐ Assign Task
- ☐ Assign Subtask

- ☐ Accept Assignment
- ☐ Decline Assignment

Calendar

- ☐ Start Date updates Calendar
- ☐ End Date updates Calendar

Notifications

- ☐ Assignment
- ☐ Acceptance
- ☐ Decline
- ☐ Comments
- ☐ Mentions
- ☐ Upcoming End Date

Attachments

- ☐ Upload File
- ☐ Upload Image
- ☐ Upload Document
- ☐ Add Reference URL

Language

- ☐ English
- ☐ French

Regression

- ☐ Existing functionality has not been broken.

Module 1 Definition of Done

Module 1 is considered complete when:

- Every user has a fully functional personal workspace.
- Users can manage tasks and subtasks.
- Users can assign work with an acceptance workflow.
- Calendar synchronization is automatic.
- Notifications work correctly.
- File attachments and reference links function correctly.
- English and French translations are complete.
- Existing workflows remain stable.
- Regression testing passes.
- Product Owner approves the implementation.

Module 2 — Weekly Accountability

The operational heartbeat of Operations OS.

Introduction

While Module 1 — Personal Work Management — provides users with a workspace to create and manage their daily work, Weekly Accountability ensures that all work is planned, reviewed, tracked, reported, and carried forward through a structured weekly operational cycle.

This module establishes the rhythm of Future Studio. Every operational week begins with a Weekly Standup and concludes with a Weekly Retro. Between these two events, users execute their work. At the end of the week, they review completed tasks, explain unfinished work, raise blockers where necessary, and automatically carry unfinished work into the following operational week.

Unlike traditional project management systems where reporting is manual, Weekly Accountability transforms everyday work into structured operational reporting. The objective is to eliminate duplicate reporting while maintaining transparency, accountability, and historical accuracy.

This module consists of six tightly integrated features:

- Weekly Standups
- Weekly Retros
- Blocker Management
- Carry-over Tasks
- Weekly Reports
- Operational Metrics

Team Note: These features form one complete operational workflow and must be developed by the same engineering team. They should not be separated, because changes in one directly affect the others.

Business Objective

The objective of Weekly Accountability is to ensure that every member of Future Studio:

- Plans their work at the beginning of every operational week.
- Executes work throughout the week.
- Reviews progress before closing the week.
- Explains unfinished work.
- Raises blockers whenever work cannot be completed.
- Carries unfinished work into the following week.
- Automatically generates accurate operational reports.

The system should eliminate repetitive reporting while increasing visibility into operational performance.

Engineering Team Instructions

This module already exists within the platform. Your responsibility is not to redesign the workflow.

Before implementing any changes:

- Review the current implementation.
- Understand how Weekly Standups currently work.
- Understand how Weekly Retros currently work.
- Review the current blocker implementation.
- Review carry-over behaviour.
- Review weekly reporting.
- Identify completed and missing functionality.
- Extend existing functionality rather than rebuilding it.

Do not redesign the user interface. Reuse existing pages, components, APIs, services, and database structures. Every implementation must support both English and French.

Relationship With Other Modules

Relationship with Module 1 — Personal Work Management

Weekly Accountability depends entirely on Module 1. Every task reviewed in Weekly Retro originates from Personal Work Management. Weekly Standups organize those tasks. Weekly Retros review those tasks. Carry-over Tasks preserve those tasks.

Relationship with Module 3 — Project Operations

Project Tasks automatically appear inside Weekly Standups. Project progress depends upon Weekly Accountability. Project Reports should consume data generated by Weekly Standups and Weekly Retros. Users should never duplicate reporting.

Relationship with Module 4 — Communication & Collaboration

When blockers are raised, Super Admins and supervisors communicate directly with the blocker owner. Conversations remain attached to operational records.

Relationship with Module 5 — Operational Administration

Operational dashboards, Weekly Reports, Productivity Metrics, Management Reports, and Executive Dashboards should all be generated automatically from Weekly Accountability.

Business Case Study

The following case study represents the official Future Studio weekly operational workflow. Developers should implement this workflow exactly as described.

Monday Morning

Sarah logs into ImpactOS. Because this is the beginning of a new operational week, the system checks whether she already has a Weekly Standup. If no Weekly Standup exists, the dashboard displays Create Weekly Standup. Sarah clicks the button and a Weekly Standup is created for the current operational week. Immediately afterwards, the Create Weekly Standup button disappears. Whenever Sarah returns during the same week, the existing Weekly Standup opens automatically.

Business Rule: One User = One Weekly Standup = One Operational Week. Duplicate Weekly Standups must never exist.

Planning the Week

Inside the Weekly Standup, Sarah begins planning her work. She creates the task “Permission Management Improvements” with Title, Description, Category, Priority, Start Date, and End Date, then creates several subtasks. Some remain assigned to herself; others are assigned to David. David receives a Pending Assignment notification and accepts the work — the subtask automatically becomes part of his workload.

Working Throughout the Week

Sarah and David continue working normally: creating additional tasks and subtasks, editing tasks, uploading files, attaching reference URLs, adding comments, and updating progress. Everything remains inside the Weekly Standup.

Reviewing the Week

On Friday, or whenever Sarah is ready to review her work, she opens Weekly Retro. The Weekly Retro automatically displays every task that belongs to that week's Weekly Standup. For every task, Sarah decides whether it has been completed or could not be completed. The Retro presents two primary actions: Mark as Completed and Add Blocker.

Raising a Blocker

Sarah could not complete the task “Integrate Resend Email Service” because the organization's DNS records have not yet been updated. Instead of marking the task completed, she selects Add Blocker. A Blocker modal opens, capturing Blocker Title, Description, Priority, an optional Reference URL, and Supporting Notes.

Example entry:

- Title: Waiting for DNS Verification
- Description: Domain verification cannot continue until DNS records are updated.
- Priority: High
- Reference URL: resend.com/domains
- Notes: Waiting for Infrastructure Team.

She saves the blocker. The task remains In Progress. The blocker is attached permanently to the task.

Supervisor Review

The Super Admin receives a notification containing the user, task, blocker title, and date raised. The Super Admin opens the blocker, reviews Sarah's explanation, and sends her a direct message requesting additional clarification — the conversation remains attached to the blocker. The Super Admin then contacts the Infrastructure Team.

Resolving the Blocker

The Infrastructure Team updates the DNS records. Sarah receives confirmation, returns to Weekly Retro, and opens the blocker, which now displays Mark Blocker as Resolved.

Business Rule: Only the user who originally created the blocker may resolve it. Super Admins, Project Managers, Department Heads, and Supervisors may all view blockers, comment, and assist, but may not resolve another user's blocker. The blocker owner remains responsible for closing the blocker.

Completing the Task

Sarah selects Mark as Completed. The system first checks for unresolved blockers. If none exist, the task completes immediately. If blockers remain, the system displays each one and asks the user to confirm whether it has been resolved. If any blocker remains unresolved, the task cannot be completed.

Parent Tasks

Sarah has another task, "Engineering Permission Refactor," with subtasks Review Existing Code ✓, Database Changes ✓, API Updates ✓, UI Testing (unresolved blocker), and Documentation ✓. Because UI Testing contains an unresolved blocker, the Parent Task cannot be completed. The system displays an explanation. Users may complete individual subtasks, but the Parent Task remains incomplete until every required subtask is complete and every blocker has been resolved.

End of Week

At the end of the week, the system automatically generates a Weekly Report, Productivity Summary, Completion Statistics, Blocker Summary, and Carry-over Summary. Sarah is not required to manually recreate information already stored inside the platform.

Beginning of the Following Week

The following Monday, Sarah logs in again and selects Create Weekly Standup. The system creates a new Weekly Standup, and every unfinished task from the previous week automatically appears as a Carry-over Task — preserving its subtasks, assignment, comments, attachments, history, and blockers. Historical Weekly Standups and Weekly Retros become read-only. Nothing is deleted.

Business Rules

13. A user may only have one Weekly Standup per operational week.
14. Every task belongs to a Weekly Standup.
15. Weekly Standup is the user's operational workspace for the week.
16. Weekly Retro is the official review point for the week's work.
17. Users may create tasks throughout the week.
18. Blockers may only be created from within the Weekly Retro.
19. The Weekly Retro should present two primary actions for each task: Mark as Completed and Add Blocker.
20. Creating a blocker requires Title, Description, Priority, optional Reference URL, and Supporting Notes.
21. A task may have multiple blockers.
22. Only the blocker owner may resolve a blocker.
23. Users with administrative privileges may view and discuss blockers but may not resolve them.
24. A task with unresolved blockers cannot be completed.
25. If a parent task contains subtasks with unresolved blockers, the parent task cannot be completed.

26. Weekly Reports are generated automatically from operational data.
27. Carry-over Tasks are generated automatically.
28. Historical Weekly Standups and Weekly Retros become read-only.
29. All interfaces, notifications, emails, and validation messages must support English and French.

Feature Breakdown

2.1 Weekly Standup	Purpose: Provide a structured workspace for planning one operational week. Includes: • Create Weekly Standup • Current Week • Previous Weeks (Read-only) • Weekly Tasks & Subtasks • Carry-over Tasks
2.2 Weekly Retro	Purpose: Review weekly work before closing the operational week. Includes: • Weekly Task Review • Completion Review • Blocker Action • Weekly Summary
2.3 Blocker Management	Purpose: Capture reasons why work could not be completed. Includes: • Blocker Button & Modal • Priority • Reference URL • Supporting Notes • Resolution Workflow • Blocker History
2.4 Carry-over Tasks	Purpose: Automatically preserve unfinished work. Includes: • Automatic Carry-over • Assignment Preservation • Attachments • Comments • Blockers • History
2.5 Weekly Reports	Purpose: Generate operational reports automatically. Includes: • Weekly Summary • Completed Tasks • Outstanding Tasks • Blockers • Carry-over • Productivity Summary

2.6 Operational Metrics

Purpose: Provide operational visibility.

Includes:

- Completion Rate
- Carry-over Rate
- Blocker Rate
- Resolution Time
- Weekly Productivity

Developer Tickets

Ticket 2.1 — Weekly Standup

Review the existing Weekly Standup implementation. Confirm the current workflow, APIs, database schema, and UI. Extend the existing functionality to enforce one Weekly Standup per operational week and automatically reopen the current week's Standup.

Acceptance Criteria

- ☐ One Weekly Standup per week.
- ☐ Duplicate Standups prevented.
- ☐ Current week's Standup reopens automatically.
- ☐ Previous Standups are read-only.
- ☐ English and French supported.

Ticket 2.2 — Weekly Retro

Review the Weekly Retro implementation and ensure it becomes the official weekly review interface.

Acceptance Criteria

- ☐ Weekly tasks and subtasks displayed automatically.
- ☐ Standup data reflected correctly.
- ☐ Completion workflow operational.
- ☐ English and French supported.

Ticket 2.3 — Blocker Management

Replace the existing blocker implementation with a dedicated Blocker action inside Weekly Retro.

Acceptance Criteria

- ☐ Blocker modal implemented; required fields validated.
- ☐ Optional Reference URL supported.
- ☐ Blockers attached to tasks; notifications triggered.
- ☐ Only blocker owner can resolve.
- ☐ Tasks with unresolved blockers cannot be completed.
- ☐ Parent tasks blocked when subtasks contain unresolved blockers.
- ☐ English and French supported.

Ticket 2.4 — Carry-over Tasks

Review and improve the carry-over workflow.

Acceptance Criteria

- ☐ Carry-over automatic.
- ☐ Assignment, comments, attachments, history, and blockers preserved.

Ticket 2.5 — Weekly Reports

Review automatic report generation.

Acceptance Criteria

- ☐ Weekly Reports generated automatically.
- ☐ No duplicate data entry.
- ☐ Historical reports preserved.
- ☐ English and French supported.

Ticket 2.6 — Operational Metrics

Review operational metric calculations.

Acceptance Criteria

- ☐ Completion rate, carry-over statistics, and blocker statistics accurate.
- ☐ Dashboard updates automatically.

Module 2 Self-Testing Checklist

Weekly Standup

- ☐ Create Weekly Standup.
- ☐ Prevent duplicate Standups.
- ☐ Reopen current week's Standup.
- ☐ Previous Standups read-only.

Weekly Retro

- ☐ Weekly tasks and subtasks displayed.
- ☐ Completion workflow works.
- ☐ Retro reflects Standup data.

Blockers

- ☐ Blocker modal opens; validation works.
- ☐ Optional Reference URL saved.
- ☐ Notifications sent.
- ☐ Only blocker owner resolves blocker.
- ☐ Tasks blocked until blockers resolved.
- ☐ Parent task completion prevented when subtasks contain unresolved blockers.

Carry-over

- ☐ Automatic carry-over.
- ☐ Assignments, comments, attachments, and blockers preserved.

Reports

- ☐ Weekly Report generated.
- ☐ Historical Reports preserved.

Metrics

- ☐ Completion Rate.
- ☐ Carry-over Rate.
- ☐ Blocker Rate.
- ☐ Productivity statistics.

Language

- ☐ English.
- ☐ French.

Regression

- ☐ Existing workflows remain operational.
- ☐ No duplicate business logic introduced.

Module 2 Definition of Done

Module 2 is considered complete when:

- Every user can create exactly one Weekly Standup per operational week.
- Weekly Standup and Weekly Retro function as one continuous operational workflow.
- Users can review work and raise blockers from within Weekly Retro.
- Only the blocker owner can resolve a blocker.
- Tasks and parent tasks cannot be completed while unresolved blockers exist.
- Carry-over Tasks are generated automatically and preserve all related data.
- Weekly Reports and Operational Metrics are generated automatically from user activity.
- All user-facing interfaces support English and French.
- Existing functionality has been preserved.
- Regression testing passes.
- The Product Owner confirms that the implementation matches the agreed Future Studio operational model.

Module 3 — Project Operations

The collaborative execution layer of Operations OS.

Introduction

While Module 1 focuses on an individual's daily work and Module 2 focuses on weekly accountability, Module 3 focuses on how multiple people work together to deliver a project.

A project is not merely a collection of tasks. It is a structured body of work with defined objectives, owners, timelines, team members, deliverables, and progress tracking. Every project should provide management with complete visibility into who is working, what is being worked on, current progress, outstanding work, risks, blockers, and overall project health.

Project Operations should integrate seamlessly with every other Operations OS module.

Business Objective

The objective of Project Operations is to provide a centralized environment where teams can:

- Create projects.
- Assign team members.
- Create project tasks.
- Assign responsibilities.
- Track progress.
- Monitor timelines.
- Identify risks.
- Produce project reports.

Projects should become the primary method through which collaborative work is coordinated inside Future Studio.

Relationship With Other Modules

Relationship with Module 1 — Personal Work Management

Every Project Task ultimately becomes someone's personal task.

Relationship with Module 2 — Weekly Accountability

Every Project Task appears inside the assigned user's Weekly Standup. Project work should naturally become part of the Weekly Retro. Carry-over Tasks should continue belonging to the same project. Project reporting should automatically include Weekly Accountability data.

Business Case Study

Creating a Project

The Engineering Supervisor decides to begin work on a new initiative: "Engineering Permission Refactor." He creates a new project containing a Project Name, Description, Owner, Department, Start Date, End Date, Status, Priority, Project Members, and Supporting Documents.

The supervisor invites Sarah, David, John, and Grace to become project members. Every invited member receives a notification. Once accepted, the project appears on each member's dashboard.

Planning Project Work

The supervisor creates several Project Tasks — Permission Profiles, Engineering Dashboard, Error Logs — each assigned to one or more project members. Each Project Task automatically becomes both a Project Task and a Personal Task inside the assigned user's Weekly Standup. The assigned user should never recreate the task manually.

Task Execution

Throughout the project, members continue working normally. Everything they do inside Personal Work Management automatically contributes to Project Operations:

- Completing Tasks → Project Progress updates.
- Completing Subtasks → Project Completion Percentage updates.
- Adding Comments → Project Activity updates.
- Uploading Files → Project Documents update.
- Creating Blockers → Project Risk Status updates.

Weekly Reporting

At the end of the week, Project Managers should not manually prepare reports. Instead, Project Reports should be generated automatically from Weekly Standups, Weekly Retros, Completed Tasks, Carry-over Tasks, Blockers, and Project Progress. The goal is to eliminate duplicate reporting.

Project Completion

When every required task has been completed and every blocker has been resolved, the Project Manager may close the project. Closing a project should lock the project, preserve historical records, prevent new task creation, and maintain all reports.

Business Rules

Project Creation

Projects must contain: Name, Description, Owner, Start Date, End Date, Status, Priority.

Project Members

Projects may contain multiple members. Users may belong to multiple projects simultaneously. Removing a member from a project should not delete historical work.

Project Tasks

Every Project Task should automatically appear as a Personal Task for the assigned user. Users should never duplicate work manually.

Progress Tracking

Project Progress should calculate automatically from Completed Tasks, Completed Subtasks, Outstanding Tasks, Outstanding Blockers, and Carry-over Tasks.

Project Reports

Project Reports should generate automatically. Users should never duplicate information already entered elsewhere.

Project History

Every project should preserve Task History, Assignment History, Comments, Files, Reports, and Completion Records.

Multilingual Support

All Project interfaces must support English and French.

Feature Breakdown

3.1 Project Management	Purpose: Create and manage projects. Includes: • Create / Edit / Archive / Close Project • Project Status • Project Priority
3.2 Project Team Management	Purpose: Manage project members. Includes: • Invite / Remove / View Members • Project Roles • Ownership
3.3 Project Tasks	Purpose: Manage collaborative work. Includes: • Project Tasks & Subtasks • Assignment • Scheduling • Categories • Priorities
3.4 Project Timeline	Purpose: Display project schedule. Includes: • Start / End Date • Milestones • Calendar View
3.5 Progress Tracking	Purpose: Automatically calculate project completion. Includes:

	• Completion % • Task Statistics • Blockers • Carry-over • Risk Level
3.6 Project Reporting	Purpose: Generate reports automatically. Includes: • Weekly Summary • Progress Summary • Outstanding Tasks • Blockers • Completion Trends

Developer Tickets

Ticket 3.1 — Project Implementation Review

Review Project implementation. Confirm existing UI, existing APIs, and existing database. Extend rather than rebuild.

Acceptance Criteria

- ☐ Existing implementation reviewed and documented.
- ☐ Extension plan approved before rebuild.

Ticket 3.2 — Project Member Workflow

Review Project Member workflow. Improve invitation and assignment.

Acceptance Criteria

- ☐ Invitation workflow functions correctly.
- ☐ Assignment respects project permissions.

Ticket 3.3 — Project Task Workflow

Review Project Task workflow. Ensure Project Tasks synchronize with Personal Work Management. No duplicate task creation.

Acceptance Criteria

- ☐ Project Tasks sync automatically with Personal Tasks.
- ☐ No duplicate task records created.

Ticket 3.4 — Timeline

Review Timeline. Synchronize with Calendar.

Acceptance Criteria

- ☐ Timeline reflects Start/End Date.
- ☐ Calendar synchronization confirmed.

Ticket 3.5 — Progress Calculations

Review Progress calculations. Replace manual calculations with automatic updates.

Acceptance Criteria

- ☐ Completion % calculates automatically.
- ☐ No manual overrides required.

Ticket 3.6 — Project Reports

Review Project Reports. Generate reports automatically from operational data.

Acceptance Criteria

- ☐ Reports generated without manual data entry.
- ☐ Historical reports preserved.

Module 3 Self-Testing Checklist

Project Creation

- ☐ Create Project
- ☐ Edit Project
- ☐ Archive Project
- ☐ Close Project

Project Members

- ☐ Add Member
- ☐ Remove Member
- ☐ Multiple Projects per User

Project Tasks

- ☐ Create Task
- ☐ Assignment
- ☐ Synchronization

Timeline

- ☐ Start Date
- ☐ End Date
- ☐ Calendar

Progress

- ☐ Completion %
- ☐ Blockers
- ☐ Carry-over

Reports

☐ Automatic Reports

☐ Historical Reports

Language

☐ English

☐ French

Regression

☐ Existing functionality remains operational.

Module 3 Definition of Done

- Projects can be created, managed, and completed.
- Project Tasks automatically synchronize with Personal Work Management.
- Weekly Accountability data automatically contributes to Project Reports.
- Progress calculations are automatic and accurate.
- Historical project data is preserved.
- English and French support are complete.
- Regression testing passes.
- Product Owner approves the implementation.

Module 4 — Communication & Collaboration

The communication layer of Operations OS.

Introduction

Communication & Collaboration provides the communication layer for Operations OS. Its purpose is to ensure that every staff member can communicate efficiently without leaving ImpactOS.

Communication should always be contextual. Rather than simply providing a chat application, the system should allow conversations to be linked naturally to work being performed. This module should reduce communication fragmentation by eliminating unnecessary conversations outside the platform.

Business Objective

Provide a secure internal communication environment where staff members can:

- Communicate individually.
- Collaborate on projects.
- Communicate within operational groups.
- Receive announcements.
- Receive operational notifications.

Communication should always support the operational workflow rather than becoming a separate activity.

Relationship With Other Modules

Module 1 — Personal Work Management

Users can discuss tasks. Task discussions remain attached to the relevant task.

Module 2 — Weekly Accountability

Supervisors can discuss blockers directly with the blocker owner. When a blocker is raised, the Super Admin can immediately message the owner for clarification without leaving the platform.

Module 3 — Project Operations

Project members communicate within the context of the project. Announcements can be made to all project members simultaneously.

Business Case Study

Sarah has raised a blocker in her Weekly Retro explaining that she cannot complete the integration of the email notification service because the required DNS records have not yet been configured.

The Super Admin receives a notification indicating that a new blocker has been raised. Instead of calling Sarah or sending messages through an external application, the Super Admin opens the blocker and sends Sarah a direct message. The message remains attached to the blocker for future reference.

Sarah replies with additional information and includes a reference link to the external DNS provider. After the infrastructure team completes the DNS configuration, Sarah receives another message informing her that the issue has been resolved. Sarah returns to the Weekly Retro, marks the blocker as resolved, and then completes the task.

Throughout this process, all communication remains attached to the operational record, management has complete visibility, no operational information is lost, and the complete history is preserved.

Another Example

Sarah is working on Project Alpha. She needs David's assistance. Instead of sending a separate email, she opens the project, opens the task, and starts a discussion attached directly to that task. Future conversations about the task remain available to every authorized team member.

Business Rules

Direct Messaging

Every authenticated internal user may send direct messages to another authorized internal user.

Task Conversations

Every task should support its own discussion thread. All conversations remain permanently attached to that task.

Project Conversations

Every project should support collaborative discussions. Project conversations remain available throughout the project's lifetime.

Blocker Conversations

Every blocker should support discussion, allowing management and staff to collaborate while resolving operational issues.

Broadcast Messages

Broadcast messaging is restricted. Only users with appropriate permissions may broadcast messages — for example, Super Admin, Program Managers, Project Owners, and Department Leads where authorized. Ordinary staff members should not broadcast messages to entire groups unless granted permission.

Announcements

Announcements should be available for the Entire Organization, Specific Groups, Specific Projects, Specific Programs, or Specific Departments. Announcements remain visible until archived.

Read Status

Messages should support Sent, Delivered, and Read states where technically feasible.

Attachments

Messages should support Documents, Images, and Reference URLs.

Notifications

Communication events should trigger notifications — for example New Message, New Broadcast, Mention, New Announcement, Reply, and Project Discussion.

Multilingual Support

Every communication interface must support English and French.

Feature Breakdown

4.1 Direct Messaging	Purpose: Allow users to communicate privately. Includes: • Send Message • Reply • Read Status • Attachments • Search Conversation
4.2 Task Discussions	Purpose: Keep communication attached to work. Includes: • Task Comments • Task Discussion History • Mentions • Attachments
4.3 Project Discussions	Purpose: Enable project collaboration. Includes: • Project Conversations • Team Discussion • Project Attachments • Project Mentions
4.4 Broadcast Messaging	Purpose: Communicate with large audiences. Includes: • Broadcast to Organization / Groups / Projects / Programs • Access controlled by permissions
4.5 Announcements	Purpose: Publish operational information. Includes: • Create / Archive / Pin Announcement • Announcement History
4.6 Notification Centre	Purpose: Provide one centralized notification hub. Includes: • Task, Assignment, Message, Announcement, Blocker, and Project Notifications

Developer Tickets

Ticket 4.1 — Direct Messaging

Review Direct Messaging implementation. Confirm existing database, UI, and APIs. Improve where necessary.

Acceptance Criteria

- ☐ Messaging functions correctly across all confirmed surfaces.
- ☐ No regressions in existing conversations.

Ticket 4.2 — Task Discussions

Implement Task Discussions. Every task should maintain its own communication history.

Acceptance Criteria

- ☐ Every task supports a discussion thread.
- ☐ History persists correctly.

Ticket 4.3 — Project Discussion Workflow

Review Project Discussion workflow. Ensure discussions remain attached to projects.

Acceptance Criteria

- ☐ Project discussions remain scoped to their project.
- ☐ Attachments preserved.

Ticket 4.4 — Broadcast Implementation

Review Broadcast implementation. Ensure broadcasts are controlled through the permission system. Do not hardcode roles.

Acceptance Criteria

- ☐ Broadcast access enforced via the centralized permission engine.
- ☐ Unauthorized users cannot broadcast.

Ticket 4.5 — Announcements

Review Announcements. Allow announcements to target Groups, Projects, Programs, and the Organization.

Acceptance Criteria

- ☐ Audience targeting functions correctly.
- ☐ Archived announcements no longer appear as active.

Ticket 4.6 — Notification Centre

Review Notification Centre. Integrate notifications with Tasks, Projects, Blockers, Messaging, and Weekly Accountability.

Acceptance Criteria

- ☐ Notifications generated across all integrated sources.
- ☐ Notification Centre reflects real-time state.

Module 4 Self-Testing Checklist

Direct Messaging

- ☐ Send message
- ☐ Reply
- ☐ Read status

- ☐ Attachment upload

Task Discussions

- ☐ Discussion created
- ☐ Comments stored
- ☐ Mentions notify users

Project Discussions

- ☐ Messages remain attached to project
- ☐ Attachments preserved

Broadcasts

- ☐ Authorized users can broadcast
- ☐ Unauthorized users cannot broadcast

Announcements

- ☐ Create announcement
- ☐ Archive announcement
- ☐ Audience targeting

Notifications

- ☐ New message notification
- ☐ Mention notification
- ☐ Announcement notification
- ☐ Blocker notification

Language

- ☐ English
- ☐ French

Regression

- ☐ Existing communication workflows remain operational.

Module 4 Definition of Done

- Internal communication is fully integrated into Operations OS.
- Users communicate without leaving the platform.
- Conversations remain attached to their operational context (tasks, blockers, or projects).
- Broadcasts and announcements follow the permission model.
- Notifications function correctly across all communication events.
- English and French support are complete.
- Regression testing passes.
- Product Owner approves the implementation.

Module 5 — Operational Administration

The administrative and management layer of Operations OS.

Introduction

While the previous modules focus on helping users perform their daily work, this module provides supervisors, managers, and administrators with the tools required to oversee the organization's operations.

Purpose: Not to micromanage users — to provide visibility, governance, operational control, and organizational administration while ensuring that staff can continue working efficiently.

This module integrates with every other module inside Operations OS.

Business Objective

Provide authorized personnel with the ability to:

- Manage teams.
- Manage internal operations.
- Monitor operational performance.
- Review organizational reports.
- Manage organizational documents.
- Monitor operational finances.
- Maintain operational oversight.

Every administrative action should improve organizational visibility without disrupting normal staff workflows.

Relationship With Other Modules

Operational Administration depends upon every previous module:

- Module 1 provides visibility into: Staff workload, Personal productivity, Task completion.
- Module 2 provides visibility into: Weekly Standups, Weekly Retros, Carry-over Tasks, Blockers, Weekly Reports.
- Module 3 provides visibility into: Projects, Project Progress, Project Health, Team Performance.
- Module 4 provides visibility into: Internal communication, Broadcasts, Announcements, Collaboration history.

Business Case Study

The Engineering Manager logs into ImpactOS on Monday morning. Unlike an intern or a regular staff member, the Engineering Manager has additional administrative responsibilities. After logging in, the manager can view Team activity, Active projects, Outstanding blockers, Weekly reports awaiting review, Recent announcements, and Department operational metrics.

The manager notices that several blockers have been raised. Rather than resolving them directly, the manager reviews the blocker details and communicates with the relevant staff members to understand what assistance is required. The manager also reviews the Engineering team's weekly productivity and identifies projects that require

additional support. No duplicate reports need to be requested, because all operational data has already been collected automatically through the normal workflows of Operations OS.

Finance Example

A Finance Officer logs into ImpactOS. Because Finance has been assigned as one of the user's responsibilities, the Finance sidebar becomes available. The officer can view financial records, review departmental expenses, upload supporting financial documents, and generate financial summaries. The Finance Officer does not automatically gain access to Engineering, Projects, or Program Management unless those responsibilities have also been assigned.

Department Administration Example

A Marketing Supervisor logs in. Their default workspace remains the Staff dashboard. However, because they have been assigned responsibility for Marketing Operations, they also see the Marketing sidebar. They can manage only the operational areas for which they have been granted access. This demonstrates that responsibilities extend a user's workspace without changing their base user type.

Business Rules

Administrative Visibility

Administrative users should only see information they are authorized to access. Access must always be determined by the centralized permission system. Do not hardcode access rules.

Team Management

Authorized users should be able to view team members, assign supervisors, review workload, and monitor operational activity.

Internal Reporting

Reports should be generated automatically from operational data. Managers should not manually recreate information that already exists.

Finance

Finance should operate as a responsibility-based module. Users without Finance responsibility should not see Finance navigation.

Documents

Operational documents should remain attached to the relevant entity — Projects, Tasks, Weekly Reports, Financial Records. Documents should not exist in isolation.

Operational Metrics

Management dashboards should update automatically — for example Weekly Completion Rate, Outstanding Blockers, Average Task Completion Time, Team Productivity, and Project Health.

Multilingual Support

Every administrative interface must support English and French.

Feature Breakdown

5.1 Team Management	Purpose: Manage organizational teams. Includes: • Team Directory • Supervisors • Team Members • Team Activity • Team Workload
5.2 Internal Reporting	Purpose: Provide operational reports. Includes: • Weekly Reports • Department Reports • Team Reports • Operational Reports
5.3 Operational Metrics	Purpose: Provide organizational visibility. Includes: • Completion Rate • Productivity • Carry-over Rate • Blocker Rate • Project Performance
5.4 Finance	Purpose: Provide operational financial visibility. Includes: • Expense Tracking • Financial Documents • Department Expenses • Financial Reports
5.5 Document Management	Purpose: Manage operational documents. Includes: • Upload / Download Files • Version History • Reference Links • Attachments

Developer Tickets

Ticket 5.1 — Team Management

Review Team Management implementation. Confirm existing UI, APIs, and database. Extend existing functionality.

Acceptance Criteria

- ☐ Existing team-management functionality reviewed and documented.
- ☐ Extension plan approved before rebuild.

Ticket 5.2 — Internal Reporting

Review Internal Reporting. Ensure reports are generated automatically from operational data.

Acceptance Criteria

- ☐ Reports generate without manual recreation of data.
- ☐ Historical reports preserved.

Ticket 5.3 — Operational Metrics

Review Operational Metrics. Verify calculations. Remove duplicate manual calculations.

Acceptance Criteria

- ☐ Metric calculations verified against source data.
- ☐ No manual calculation steps remain.

Ticket 5.4 — Finance

Review Finance implementation. Ensure Finance visibility follows the centralized permission engine. Do not hardcode user roles.

Acceptance Criteria

- ☐ Finance navigation only visible to authorized users.
- ☐ Access enforced via the permission engine.

Ticket 5.5 — Document Management

Review Document Management. Ensure every uploaded document remains attached to its parent entity.

Acceptance Criteria

- ☐ Documents remain linked to Projects, Tasks, Reports, or Financial Records as applicable.
- ☐ Version history preserved.

Module 5 Self-Testing Checklist

Team Management

- ☐ View Teams
- ☐ View Members
- ☐ Supervisor Assignment
- ☐ Workload Review

Internal Reports

- ☐ Weekly Reports
- ☐ Team Reports
- ☐ Department Reports

Operational Metrics

- ☐ Completion Rate

- ☐ Productivity
- ☐ Carry-over
- ☐ Blockers
- ☐ Project Health

Finance

- ☐ Permission-controlled visibility
- ☐ Expense records
- ☐ Financial reports

Documents

- ☐ Upload
- ☐ Download
- ☐ Version history
- ☐ Parent relationship preserved

Language

- ☐ English
- ☐ French

Regression

- ☐ Existing functionality remains operational.

Module 5 Definition of Done

- Administrative users can oversee organizational operations through centralized dashboards.
- Team management, reporting, metrics, finance, and document management integrate with the existing platform.
- Access is governed entirely through the centralized permission engine.
- Documents remain linked to their parent records.
- All interfaces support English and French.
- Regression testing passes.
- Product Owner approves the implementation.

Cross-Module Integration

Operations OS is not a collection of independent modules. Every module depends on the others. The operational flow is:

User Logs In
Personal Work Management (Module 1)
Weekly Standup (Module 2)
Tasks & Subtasks
Project Operations (Module 3)
Communication & Collaboration (Module 4)
Weekly Retro (Module 2) — Completed → Weekly Reports Blocker → Supervisor Review → Resolution
Operational Administration (Module 5)
Executive Reporting & Metrics

Core Principle: Every action taken by a user should contribute automatically to organizational reporting. Users should never be asked to enter the same information twice.

General Engineering Principles

Every implementation within Operations OS must follow these principles:

- 30. Understand the existing implementation before modifying it.
- 31. Extend existing functionality before creating new functionality.
- 32. Avoid duplicate APIs, components, services, or business logic.
- 33. Build configurable solutions rather than hardcoded workflows.
- 34. Ensure every feature integrates naturally with the rest of Operations OS.
- 35. Use centralized permissions for access control.
- 36. Reuse global styles, reusable components, and utility functions.
- 37. Maintain backward compatibility.
- 38. Support both English and French in every user-facing interface.
- 39. Perform regression testing before submitting work.

Final Definition of Done — Operations OS

Operations OS will be considered complete only when:

- All five modules have been fully implemented.
- The business workflows described in this specification operate as intended.
- Tasks, projects, standups, retros, blockers, reporting, communication, and administration function as one integrated operating environment.
- No duplicate reporting or duplicate data entry is required.
- All user-facing features support both English and French.
- The implementation reuses the existing architecture wherever possible.
- Regression testing confirms that existing functionality remains stable.
- The Product Owner validates that the implemented workflows align with Future Studio's operational model.

Recommendation Before Development

Before implementation begins, each engineering team assigned to a module should:

40. Review the existing implementation.
41. Document what already exists.
42. Identify gaps between the existing implementation and this specification.
43. Present their findings for Product Owner review.
44. Receive approval before modifying business logic.
45. Implement the approved changes.
46. Complete self-testing and regression testing.
47. Submit the module for Product Owner validation.

Sequencing: Only after Operations OS has been validated end-to-end should development proceed to Program OS, ensuring that the operational foundation of ImpactOS is stable before expanding into the remaining Business Operating Systems.